

BUILD SPECIFICATION / PART 1 OF 2 // v1.0 // SEPTEMBER 2026

High-Level Design

Agency Portfolio Website

Architecture, scope, stack, design direction and delivery plan

Contents

Contents	2
1. Agent Brief (Read First)	3
2. Product Overview	3
3. Users and Key Journeys	4
4. Information Architecture	4
5. System Architecture	5
6. Technology Stack	6
7. Feature Overview	6
8. UX and Visual Direction	7
9. Data Overview	8
10. Security and Privacy	8
11. Non-Functional Requirements	8
12. Environments and Deployment View	9
13. Delivery Plan, Risks and Assumptions	9

1. Agent Brief (Read First)

This document pair (High-Level Design + Low-Level Design) is a complete specification for building an **agency portfolio website** from an empty folder to a live production deployment. The agency designs and builds websites for clients; this site is its own storefront, portfolio and lead-generation engine.

- **Build everything from scratch** in a new monorepo with two apps: `client/` (React) and `server/` (Node.js + Express). No paid UI kits or purchased templates; every component is hand-built to the design system.
- **Read order:** this HLD first (what and why), then the LLD (exact folders, tokens, schemas, endpoints, component specs, deployment runbook). If they ever conflict, the LLD wins on implementation detail.
- **Do not stop to ask about placeholders.** Brand name, copy, prices, team and contact details are placeholders held in `client/src/config/site.js` and `client/src/content/*.js`. Use the defaults given and make them trivial to edit.
- **Work phase by phase** (LLD section 14). After each phase run lint, tests and build, and verify that phase's acceptance criteria before moving on.
- **Deployment needs owner-created accounts and secrets** (MongoDB Atlas, Cloudinary, SMTP email, Render, Vercel). Prepare every config file and script first, then hand the owner one exact numbered checklist for the manual steps, and complete the remaining steps once secrets are supplied.
- **Final deliverables:** GitHub-ready repository, README, `.env.example` files, live frontend URL, live API URL, seeded admin login, and a short handover note.

2. Product Overview

2.1 Purpose

A premium, dark-themed marketing and portfolio website for a web-development agency. It must prove quality through visible work, explain the offer clearly, show transparent pricing, and convert visitors into inquiries - while looking distinctly more polished than typical agency templates.

2.2 Goals and success measures

ID	Goal	Success measure
G1	Showcase work	Home page lists all published projects from the API, with category filters and full case-study pages
G2	Build trust	About page (story, values, team) plus testimonials and stats on Home
G3	Transparent pricing	Pricing page with 3 tiers, comparison table, add-ons and FAQ
G4	Generate leads	Contact form stores inquiries in the database and emails the agency; a call-to-action on every page
G5	Self-serve content	Admin can create, edit, publish and unpublish projects and read inquiries with no code changes
G6	Distinctive UI	Dark premium theme with a signature aesthetic and a strict, consistent design system
G7	Production ready	Lighthouse targets met, hardened API, CI pipeline, deployed on managed hosting with HTTPS

2.3 Scope

In scope (v1)	Out of scope (v1)
Home, About, Pricing, Contact, Project detail and 404 pages Dark theme with design system and motion Public REST API for projects and inquiries Admin login, dashboard, project CRUD with image upload, inquiry inbox SEO (meta, Open Graph, JSON-LD, sitemap), accessibility, performance work CI pipeline and production deployment	Blog / articles Payments or checkout Client login portal Multiple languages Light theme toggle Live chat Drag-and-drop reordering (numeric order field is used instead)

3. Users and Key Journeys

Persona	Needs	Primary journey
Prospective client (founder / SMB owner)	Judge quality fast, understand cost, feel safe hiring	Home (hero + work) > Project detail > Pricing > Contact form
Agency admin (staff)	Publish new work, manage leads without a developer	Admin login > New project > Upload images > Publish > Check inquiries
Recruiter / partner	Understand the team and capabilities	About > Home (work) > Contact

4. Information Architecture

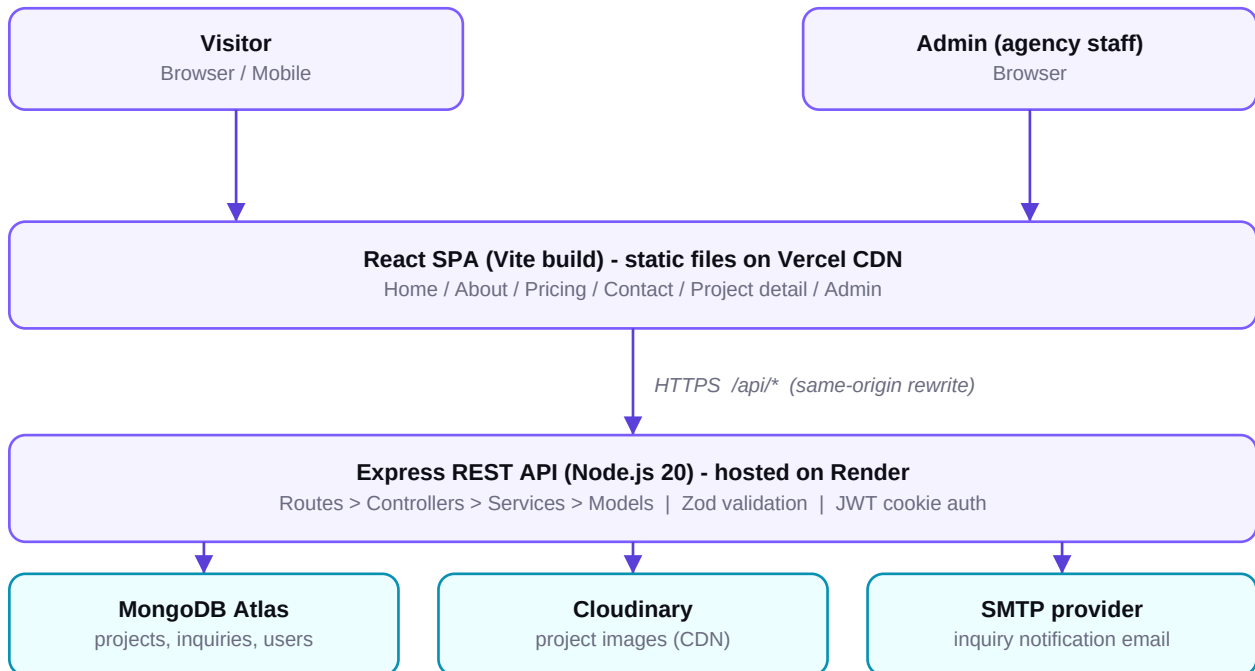
4.1 Sitemap

Route	Page	Access	Notes
/	Home	Public	Hero, marquee, stats, services, Our Work (published projects) , process, testimonials, pricing teaser, CTA
/about	About	Public	Story, mission, values, team, milestones, tech stack
/pricing	Pricing	Public	Three plans, comparison, add-ons, FAQ
/contact	Contact	Public	Project inquiry form and contact details
/projects/:slug	Project detail	Public	Case study for one published project
/admin/login	Admin login	Public (noindex)	Email + password
/admin	Dashboard	Admin	Counts and latest inquiries
/admin/projects	Project manager	Admin	List, filter, publish toggle, edit, delete
/admin/projects/new and /:id/edit	Project form	Admin	Create and edit with image upload
/admin/inquiries	Inquiry inbox	Admin	Read and change status
*	404	Public	Branded not-found page

4.2 Navigation

Public navbar: **Home, Work, About, Pricing, Contact** plus a primary button **Start a project**. "Work" scrolls to the Our Work section on Home ([/#work](#work)). The footer repeats navigation, services and contact details.

5. System Architecture



The frontend is a static single-page app served from a CDN. The backend is a stateless REST API. A same-origin rewrite (`/api/*` on the frontend domain proxied to the API host) removes cross-site cookie and CORS pain.

5.1 Components

Component	Responsibility
React SPA (Vite)	All UI, routing, animation, forms, SEO tags, calls the API
Vercel	Hosts static build, HTTPS, preview deployments, <code>/api/*</code> rewrite to the API host
Express API (Render)	Validation, business logic, auth, rate limiting, persistence, media upload, email dispatch
MongoDB Atlas	Stores projects, inquiries and admin users
Cloudinary	Stores and optimises project images; serves them from a CDN with auto format and quality
SMTP provider	Sends inquiry notification (and optional auto-reply) emails
GitHub + Actions	Source control, CI (lint, test, build); Vercel and Render auto-deploy from the main branch

5.2 Key request flows

- Browse:** browser loads `/` > CDN returns static files > Home calls `GET /api/projects` > API reads indexed Mongo collection > JSON envelope returned with cache headers > project grid renders.
- Inquire:** visitor submits form > client validates (Zod) > `POST /api/inquiries` > server re-validates, checks honeypot and rate limit > saves > sends email without blocking > returns 201 > success state and toast.
- Publish:** admin logs in > server sets httpOnly JWT cookie > admin uploads image (`POST /api/admin/uploads` > Cloudinary) > saves project as draft > toggles publish > project appears on Home immediately after cache expiry.

6. Technology Stack

Layer	Choice	Reason
UI framework	React 18 + Vite 5	Fast dev server and build, route-level code splitting
Routing	React Router v6	Nested routes, lazy loading, admin route guard
Styling	Tailwind CSS 3.4 + CSS variable tokens	One source of truth for colours, spacing, radius; small CSS output
Motion	Framer Motion	Scroll reveals, layout animations, page transitions
Smooth scroll	Lenis	Premium scroll feel; disabled for reduced-motion users
Server state	TanStack Query v5 + Axios	Caching, loading and error states, retries
Forms	React Hook Form + Zod	Typed validation with shared error styling
SEO	react-helmet-async	Per-route title, meta, Open Graph, JSON-LD
Icons / toasts / markdown	lucide-react, sonner, react-markdown	Consistent icon set, accessible toasts, safe case-study rendering
Runtime	Node.js 20 LTS (ES modules)	Current LTS, native fetch and --watch
API framework	Express 4	Mature, simple, well supported
Database	MongoDB Atlas + Mongoose 8	Flexible project documents, free tier available
Validation / auth	Zod, jsonwebtoken, bcryptjs	Schema validation; JWT in httpOnly cookie; hashed passwords
Hardening	helmet, cors, express-rate-limit, express-mongo-sanitize, compression	Standard secure defaults
Media / email	Cloudinary SDK + multer, Nodemailer	Image pipeline; provider-agnostic SMTP
Quality	ESLint, Prettier, Vitest, React Testing Library, Supertest, mongodb-memory-server	One test runner across client and server
Hosting / CI	Vercel, Render, GitHub Actions	Free tiers, Git-based deploys, preview URLs

Equivalent alternatives (Netlify for the client, Railway or Fly.io for the API) are acceptable only if the same-origin [/api](#) proxy is preserved.

7. Feature Overview

Area	Features
Public site	Sticky blur navbar, aurora hero with two calls-to-action, tech marquee, animated stats, bento services grid, process timeline, testimonials, pricing teaser, footer with large wordmark
Portfolio	API-driven project grid with category filter chips, load-more, skeleton and empty states, case-study page with gallery, results, tech stack, testimonial and next-project link
Pricing	Three tier cards (middle highlighted), feature comparison table, add-ons list, accordion FAQ
Lead capture	Validated contact form with project type and budget, honeypot spam trap, rate limiting, database record, email notification
Admin	Secure login, dashboard counts, project CRUD, draft/published toggle, image upload with preview, inquiry inbox with status

Area	Features
Platform	SEO tags and sitemap, WCAG 2.1 AA accessibility, responsive 320px to 1920px, code splitting, CI, health endpoint

8. UX and Visual Direction

Concept: "Midnight Aurora". A near-black canvas with restrained violet-to-cyan light. It should feel like a premium product studio, not a template.

8.1 Design principles

- **Dark, not black.** Layered surfaces (base, elevated, card) create depth instead of flat black.
- **Light is the accent.** Gradient glow is reserved for emphasis: hero, primary buttons, the featured pricing plan, hover states.
- **One grid, one rhythm.** A 12-column grid, a 4/8px spacing scale and one identical section structure across every page.
- **Typography does the heavy lifting.** Large tight display headings, monospace eyebrow labels, generous line height.
- **Motion with purpose.** Subtle reveals and hover feedback that never block interaction and fully respect reduced-motion settings.
- **Work is the hero.** Real project imagery gets the most visual weight on the page.

8.2 Signature effects (what makes it stand out)

Effect	Where used
Aurora glow orbs (blurred, slowly floating violet / cyan / pink)	Hero and CTA banner backgrounds
Faint grid lines with radial fade mask	Hero background
Fine film-grain noise overlay (about 3.5% opacity)	Whole site, fixed layer
Cursor-follow spotlight border glow on cards	Services, projects, pricing, team cards
Gradient border on the featured card	Popular pricing plan, featured project
Animated counters and infinite marquee	Stats bar, tech logos strip
Bento-grid layout with mixed tile sizes	Services section
Blur-backdrop navbar that compacts on scroll; page fade transitions; Lenis smooth scroll	Global

8.3 Consistency contract

- Every content section = **Section** wrapper + **SectionHeader** (eyebrow, title, subtitle) + content grid + optional CTA.
- Only three button variants (primary, secondary, ghost) and three sizes; all cards use the single **Card** component.
- One icon library (lucide) at 20px or 24px with 1.75 stroke; one radius scale; one spacing scale; one type scale.
- All colours, fonts, radii and shadows come from tokens. Hard-coded hex values in components are not allowed.

9. Data Overview

Entity	Stored in	Purpose	Key fields
Project	MongoDB	Portfolio item and case study	title, slug, summary, description, category, techStack, coverImage, gallery, liveUrl, results, featured, status, order
Inquiry	MongoDB	Lead from contact form	name, email, company, projectType, budget, message, status
User	MongoDB	Admin account	email, passwordHash, name, role
Site content	Code (<code>content/*.js</code>)	About, services, pricing, FAQ, team, testimonials	Static, edited in code, version controlled

Pricing, team, services and FAQ are deliberately static content files: they change rarely, ship faster and remove the need for extra admin screens. Projects are dynamic because the agency adds them continuously.

10. Security and Privacy

- Passwords hashed with bcrypt (cost 12). Admin auth uses a JWT stored in an httpOnly, Secure, SameSite=Lax cookie; nothing sensitive in localStorage.
- All input validated with Zod on the server; Mongo operator injection blocked with `express-mongo-sanitize`; request body limited to 100 KB.
- helmet security headers, strict CORS allowlist, rate limits (global, inquiries, login), honeypot field on the contact form.
- Image uploads restricted by MIME type and size (5 MB); stored on Cloudinary, never on the API disk.
- Secrets only in environment variables; `.env` files git-ignored; `.env.example` documents every variable; env validated at boot.
- Personal data collected is limited to what the contact form asks for; add a short privacy note under the form.

11. Non-Functional Requirements

Area	Target
Performance	Lighthouse mobile ≥ 90 ; LCP < 2.5 s; CLS < 0.1 ; TBT < 200 ms; route-level code splitting; animations use transform and opacity only
Accessibility	WCAG 2.1 AA; Lighthouse ≥ 95 ; visible focus rings; skip link; keyboard operable menu, filters and accordion; reduced-motion respected; text contrast $\geq 4.5:1$
SEO	Lighthouse ≥ 95 ; unique title and description per route; Open Graph and Twitter cards; JSON-LD; sitemap.xml and robots.txt
Responsive	320px to 1920px; mobile-first; no horizontal scroll at any width
Browsers	Last two versions of Chrome, Edge, Firefox, Safari; iOS Safari 16+
Reliability	<code>GET /api/health</code> ; graceful loading, empty and error states; email failure never loses an inquiry
Maintainability	ESLint + Prettier, tests, README, validated env config, clear folder conventions
Scalability	Stateless API, indexed queries, static assets on CDN, image transforms offloaded to Cloudinary

12. Environments and Deployment View

Environment	Frontend	API	Database
Local	Vite on :5173 with proxy /api to :5000	Node on :5000 (<code>npm run dev</code>)	Atlas dev database or local MongoDB
Preview	Vercel PR preview URL	Shared staging API (optional)	Atlas staging database
Production	Vercel (custom domain, HTTPS)	Render web service	Atlas production cluster

CI runs lint, tests and build on every push and pull request. Merging to `main` triggers automatic production deploys on Vercel and Render. The exact runbook is in LLD section 15.

13. Delivery Plan, Risks and Assumptions

Phase	Outcome
P0 Scaffold	Monorepo, tooling, lint, env validation, CI skeleton
P1 Design system	Tokens, Tailwind config, UI components, Navbar, Footer, aurora and noise, hidden style-guide route
P2 Backend core	Models, public API, inquiries, email, seed, tests
P3 Public pages	Home, About, Pricing, Contact, Project detail, 404 wired to the API
P4 Admin	Auth, dashboard, project CRUD with uploads, inquiry inbox
P5 Polish	SEO, accessibility, performance, empty and error states, responsive QA
P6 Quality	Automated tests, Lighthouse checks, cross-browser pass
P7 Ship	Deploy, domain, smoke tests, handover note

Risk	Mitigation
Free API tier sleeps and causes a slow first request	Uptime ping to <code>/api/health</code> every 10 minutes, or use a paid always-on instance; skeleton loaders hide latency
SPA is weaker for SEO than server rendering	Per-route meta, JSON-LD, sitemap; optional prerender step later if organic search becomes a priority
Heavy visual effects hurt performance	Transform/opacity animations only, lazy-load below-the-fold sections, cap blur layers, disable on reduced-motion
Cross-domain cookie problems	Same-origin <code>/api</code> rewrite on Vercel
Placeholder copy or prices go live by mistake	Content checklist in the handover note; all placeholders live in two folders
Large images slow the grid	Cloudinary <code>f_auto,q_auto,w_*</code> transforms, lazy loading, explicit width and height

Assumptions

- Brand name, logo, colours beyond the tokens, real copy and real project screenshots will be supplied later; generated placeholders are used now.
- Currency and price points in Pricing are sample values; they are configuration, not code.
- A single admin account (seeded) is enough for v1; multi-user roles are out of scope.

Definition of done: all pages match the design system, all acceptance criteria in LLD section 14 pass, Lighthouse targets are met on the production URL, the contact form delivers an email and a database record, and an admin can publish a project that appears on Home.